Reviewer Pack — Minimal Geometric Complexity of Tensors and Communication Ra…

Entry 45a505a4c36f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 21:53:34 UTC

Minimal Geometric Complexity of Tensors and Communication Rank

Entry ID: `45a505a4c36f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 21:53:34 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory (GCT) **Field B** (complexity object): Communication Complexity

Statement:

> For any given Boolean function f, the minimal geometric complexity of its tensor representation is linearly correlated with its communication rank, such that the geometric complexity G(f) = Θ(rank(f)). Furthermore, for all instances with a fixed communication rank, the geometric complexity is bounded by a polynomial in the number of inputs, i.e., G(f) ∈ O(n^k) for some constant k.

Rationale (proposer's reasoning):

> Geometric Complexity Theory has developed tools to study the complexity of polynomials and tensors, which are closely related to boolean functions. Communication complexity measures the amount of communication required between two parties to compute a given function. By linking these two fields, we can potentially gain insights into the structure of boolean functions that affect their communication complexity.

Taxonomy category: `Geometric Complexity Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `86a6a5950b7d2ef5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if and only if (1) Pearson correlation coefficient between geometric complexity G(f) and communication rank rank(f) exceeds 0.8, and (2) for a fixed communication rank, the mean geometric complexity across all instances with that rank is less than or equal to 3 standard deviations below the median.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Geometric Complexity Theory" AND "Communication Complexity" AND tensor representation" - "tensor representation" [ti] AND communication rank [ti]" - "minimal geometric complexity" [ti] AND polynomial bound [ti] AND Communication Complexity

Top relevant hits considered: - [http://arxiv.org/abs/astro-ph/9705063v1] NLTE effects of Ti~I in M dwarfs and giants - [http://arxiv.org/abs/2307.04643v1] MultiQG-TI: Towards Question Generation from Multi-modal Sources - [http://arxiv.org/abs/2511.19117v3] 3M-TI: High-Quality Mobile Thermal Imaging via Calibration-free Multi-Camera Cross-Modal Diffusion - [http://arxiv.org/abs/1506.04722v1] Complexity of Manipulative Actions When Voting with Ties - [http://arxiv.org/abs/1505.02399v2] Attention on Weak Ties in Social and Communication Networks - [http://arxiv.org/abs/1911.09379v2] Parameterized Complexity of Stable Roommates with Ties and Incomplete Lists Through the Lens of Graph Parameters

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def calculate_tensor_representation(f, n):
    tensor = [[Fraction(f[i * (1 << j) + k], 2**(n - j)) for k in range(1 << j)] for j in range(n)
    return tensor

def calculate_geometric_complexity(tensor):
    # Placeholder function to compute geometric complexity
    # This is a dummy implementation and should be replaced with actual GCT code
    n = len(tensor)
    return sum(sum(abs(x) for x in row) for row in tensor)

def calculate_communication_rank(f, n):
    # Placeholder function to compute communication rank
    # This is a dummy implementation and should be replaced with actual communication complexity c
    return random.randint(1, n)

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    metric_values = []
    instances_tested = 0
    n_max = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        instances_tested += n
        if instances_tested > 25:  # Avoid running too long
            break

        f = [random.randint(0, 1) for _ in range(1 << n)]
        tensor = calculate_tensor_representation(f, n)
        geometric_complexity = calculate_geometric_complexity(tensor)
        communication_rank = calculate_communication_rank(f, n)

        metric_values.append((geometric_complexity, communication_rank))
        n_max = max(n_max, n)

    if instances_tested < 30:
        return {
            "metric_name": "Geometric Complexity vs Communication Rank",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "Insufficient instances tested"
        }

    mean_geometric_complexity = sum(x[0] for x in metric_values) / len(metric_values)
    std_geometric_complexity = math.sqrt(sum((x[0] - mean_geometric_complexity)**2 for x in metric
    median_geometric_complexity = sorted([x[0] for x in metric_values])[len(metric_values) // 2]

    correlation_coefficient = sum((x[0] - mean_geometric_complexity) * (x[1] - mean_communication_
    correlation_coefficient /= instances_tested * std_geometric_complexity * std_communication_ran

    if correlation_coefficient < 0.8:
        conjecture_holds = False
        counterexample = "Correlation coefficient too low"

    return {
        "metric_name": "Geometric Complexity vs Communication Rank",
        "metric_value": mean_geometric_complexity,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_geometric_complexity = sum(x["metric_value"] for x in results if x["metric_value"] is not
    std_geometric_complexity = math.sqrt(sum((x["metric_value"] - mean_geometric_complexity)**2 fo
    support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

    if all(x["conjecture_holds"] for x in results):
        print(f"RESULT: SUPPORTED mean={mean_geometric_complexity} std={std_geometric_complexity}
    elif any(not x["conjecture_holds"] for x in results):
        first_failing_seed = next(x["seed"] for x in results if not x["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient too low\" first_failing
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad2ac5be.py", line 91, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad2ac5be.py", line 48, in run_trial
    tensor = calculate_tensor_representation(f, n)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad2ac5be.py", line 19, in calculate_ten
    tensor = [[Fraction(f[i * (1 << j) + k], 2**(n - j)) for k in range(1 << j)] for j in range(n)]
                        ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed due to a NameError, which prevented it from producing any data necessary to evaluate the conjecture. | next: Debug the error in the test code and rerun the test to provide evidence for or against the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26507
2	preregistration	ollama_remote	glm4:latest	0	0	10106
3	novelty	ollama_remote	glm4:latest	0	0	8328
4	novelty	ollama_remote	glm4:latest	0	0	12364
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19768
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	50436
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13802
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14116
9	judge	ollama_remote	glm4:latest	0	0	9684

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 165111 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/45a505a4c36f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/45a505a4c36f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/45a505a4c36f.tar.gz` (if generated)